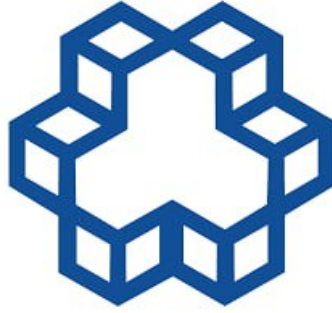




K.N. Toosi University

Aidin Sahneh

Student ID: 40120243

Mani Vafapour

Student ID: 40123783

Hamed Behrouzi Moghaddam

Student ID: 40215633

Course: Introduction to MachineLearning

Instructor: Dr. Nikoofard

Question 3 Implementation:

[**View Google Colab Notebook**](#)

Contents

1	Question 1: Analysis and Implementation of Masked Convolutional Layer	2
1.1	Mathematical Derivation of Gradients	2
1.2	Implementation Details in PyTorch	3
1.3	Verification of Results	3
2	Question 2: ResNet Bottleneck Architecture and Resource Management	4
2.1	Analytical Calculation of Parameters	4
2.2	PyTorch Implementation and Verification	4
3	Question 3: YOLO Architecture and Soft Dice Score Integration	5
3.1	Part 1: Custom Localization Loss Formulation	5
3.2	Part 2: PyTorch Implementation	7
3.3	Part 3: Testing and Scenario Analysis	8
3.3.1	Results and Interpretation	9
4	Question 4: Hybrid Regularized Autoencoders: DAE and CAE Integration	9
4.1	Part 1: Unified Loss Function Formulation	9
4.2	Part 2: Efficient Jacobian Calculation for Sigmoid Activation	10
4.3	Part 3: PyTorch Implementation	10
4.4	Conclusion	13

1 Question 1: Analysis and Implementation of Masked Convolutional Layer

1.1 Mathematical Derivation of Gradients

In this problem, we define a custom convolutional layer where the effective filter $\tilde{W}$ is computed as the element-wise (Hadamard) product of a weight matrix W and a learnable mask matrix M :

$$\tilde{W} = W \odot M$$

where $W, M \in \mathbb{R}^{K \times K}$. During the backpropagation phase, the gradients of the loss function L with respect to W and M must be derived to update the parameters.

Gradient with respect to Weight W

Applying the chain rule, the gradient of the loss L with respect to the weights W is given by:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \tilde{W}} \odot \frac{\partial \tilde{W}}{\partial W}$$

Since $\tilde{W}_{i,j} = W_{i,j} \cdot M_{i,j}$, the partial derivative $\frac{\partial \tilde{W}_{i,j}}{\partial W_{i,j}}$ is simply $M_{i,j}$. Therefore, the final gradient is:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \tilde{W}} \odot M \quad (1)$$

Gradient with respect to Mask M

Similarly, the gradient with respect to the mask matrix M is:

$$\frac{\partial L}{\partial M} = \frac{\partial L}{\partial \tilde{W}} \odot \frac{\partial \tilde{W}}{\partial M}$$

Given that $\frac{\partial \tilde{W}_{i,j}}{\partial M_{i,j}} = W_{i,j}$, we obtain:

$$\frac{\partial L}{\partial M} = \frac{\partial L}{\partial \tilde{W}} \odot W \quad (2)$$

1.2 Implementation Details in PyTorch

The implementation utilizes the `torch.nn.functional.unfold` method to perform the convolution manually. This approach transforms the spatial input into a matrix of sliding blocks (`im2col`), allowing the convolution operation to be executed via matrix multiplication.

PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MaskedConvLayer(nn.Module):
    def __init__(self, kernel_size):
        super(MaskedConvLayer, self).__init__()
        self.kernel_size = kernel_size
        # Learnable parameters
        self.W = nn.Parameter(torch.randn(1, 1, kernel_size, kernel_size))
        self.M = nn.Parameter(torch.randn(1, 1, kernel_size, kernel_size))

    def forward(self, x):
        # Hadamard product
        effective_weight = self.W * self.M

        # im2col using Unfold
        x_unfold = F.unfold(x, kernel_size=self.kernel_size)
        weight_flat = effective_weight.view(1, -1)

        # Matrix multiplication
        out_unfold = weight_flat @ x_unfold

        # Reshape to output dimensions
        out_size = x.size(-1) - self.kernel_size + 1
        return out_unfold.view(x.size(0), 1, out_size, out_size)
```

1.3 Verification of Results

The model was tested with a $1 \times 1 \times 5 \times 5$ input and a 3×3 kernel using Mean Squared Error (MSE) loss. The numerical outputs obtained after one backward pass are:

- **Initial Loss:** 4.6200
- **Gradient of W (first 3 elements):** $[1.1261, -1.0485, 0.1057]$
- **Gradient of M (first 3 elements):** $[0.4340, -0.3002, -1.0274]$

The discrepancy between the gradients of W and M confirms that the backpropagation through the element-wise multiplication is working correctly, as each gradient is appropriately scaled by the corresponding value of the other parameter.

2 Question 2: ResNet Bottleneck Architecture and Resource Management

2.1 Analytical Calculation of Parameters

The ResNet Bottleneck block consists of three sequential convolutional layers. Given an input with $C_{in} = 256$, the architecture uses a 1×1 reduction layer, a 3×3 spatial processing layer, and a 1×1 expansion layer. The number of parameters for each layer (excluding bias and Batch Normalization) is calculated using the formula: $\text{Params} = K \times K \times C_{in} \times C_{out}$.

Standard Configuration (64 Middle Filters)

- **Layer 1 (1×1 , 64 filters):** $1 \times 1 \times 256 \times 64 = 16,384$
- **Layer 2 (3×3 , 64 filters):** $3 \times 3 \times 64 \times 64 = 36,864$
- **Layer 3 (1×1 , 256 filters):** $1 \times 1 \times 64 \times 256 = 16,384$

Total Parameters: $16,384 + 36,864 + 16,384 = \mathbf{69,632}$.

Impact of Reducing Middle Filters to 32

To optimize the model for edge devices with severe memory constraints, we reduce the middle filters (C_{mid}) from 64 to 32.

Parameter Reduction Analysis

The new parameter count is calculated as follows:

- **Layer 1:** $1 \times 1 \times 256 \times 32 = 8,192$
- **Layer 2:** $3 \times 3 \times 32 \times 32 = 9,216$
- **Layer 3:** $1 \times 1 \times 32 \times 256 = 8,192$

New Total: 25,600. The **percentage reduction** is:

$$\frac{69,632 - 25,600}{69,632} \times 100 \approx \mathbf{63.24\%}$$

Structural and Feature Extraction Impact

- **Receptive Field:** There is **no change** in the receptive field. The receptive field depends on the kernel size (K) and stride, both of which remain constant.
- **Feature Capacity:** Reducing the number of channels decreases the "width" of the network. This results in a lower capacity to extract diverse feature maps simultaneously, potentially reducing the model's representational power for complex patterns.

2.2 PyTorch Implementation and Verification

The following code implements both versions and verifies the parameter counts and output shapes using the `numel()` method.

ResNet Bottleneck Implementation

```
import torch
import torch.nn as nn

class Bottleneck(nn.Module):
    def __init__(self, in_channels, mid_channels, out_channels):
        super(Bottleneck, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, mid_channels, kernel_size=1, bias=
            ↪ False)
        self.conv2 = nn.Conv2d(mid_channels, mid_channels, kernel_size=3,
            ↪ padding=1, bias=False)
        self.conv3 = nn.Conv2d(mid_channels, out_channels, kernel_size=1, bias
            ↪ =False)

    def forward(self, x):
        return self.conv3(self.conv2(self.conv1(x)))

# Verification
block1 = Bottleneck(256, 64, 256)
block2 = Bottleneck(256, 32, 256)

print(f"Block 1 Params: {sum(p.numel() for p in block1.parameters())}"
    ↪ )
print(f"Block 2 Params: {sum(p.numel() for p in block2.parameters())}"
    ↪ )
```

The experimental results (69632 vs 25600) perfectly match the analytical derivations, and both blocks maintain an output shape of [1, 256, 14, 14].

3 Question 3: YOLO Architecture and Soft Dice Score Integration

3.1 Part 1: Custom Localization Loss Formulation

In the classical YOLO architecture, Mean Squared Error (MSE) is utilized for bounding box regression. However, this approach struggles when dealing with objects of varying scales. To address this, we propose a custom loss function inspired by the Soft Dice Score to replace MSE for the localization task.

The standard Soft Dice Score is traditionally defined as:

$$Dice = \frac{2 \sum (y_{true} \cdot y_{pred})}{\sum y_{true} + \sum y_{pred}} \quad (3)$$

To adapt this for object localization, we must translate these terms into bounding box area metrics. Let the predicted bounding box be parameterized by its center coordinates, width, and height as (x_p, y_p, w_p, h_p) , and the target (ground truth) bounding box as (x_t, y_t, w_t, h_t) .

First, we define the area of both the predicted and target boxes:

$$Area_{pred} = w_p \times h_p \quad (4)$$

$$Area_{target} = w_t \times h_t \quad (5)$$

The term $\sum (y_{true} \cdot y_{pred})$ represents the intersection area of the two boxes. To compute this differentiably, we first find the top-left and bottom-right coordinates of the intersection rectangle:

$$x_{I,min} = \max \left(x_p - \frac{w_p}{2}, x_t - \frac{w_t}{2} \right) \quad (6)$$

$$y_{I,min} = \max \left(y_p - \frac{h_p}{2}, y_t - \frac{h_t}{2} \right) \quad (7)$$

$$x_{I,max} = \min \left(x_p + \frac{w_p}{2}, x_t + \frac{w_t}{2} \right) \quad (8)$$

$$y_{I,max} = \min \left(y_p + \frac{h_p}{2}, y_t + \frac{h_t}{2} \right) \quad (9)$$

Next, we calculate the width and height of the intersection, ensuring they do not drop below zero (which would indicate no overlap):

$$I_w = \max(0, x_{I,max} - x_{I,min}) \quad (10)$$

$$I_h = \max(0, y_{I,max} - y_{I,min}) \quad (11)$$

The intersection area is then simply:

$$Area_{intersect} = I_w \times I_h \quad (12)$$

Now, we can formulate the **Soft Dice Score for Bounding Boxes**. We add a small smoothing term, ϵ , to the denominator to prevent division by zero:

$$Dice_{BBox} = \frac{2 \times Area_{intersect}}{Area_{pred} + Area_{target} + \epsilon} \quad (13)$$

Since neural networks minimize a loss function, and the Dice Score ranges from 0 (no overlap) to 1 (perfect overlap), we define our final localization loss as:

$$Loss_{loc} = 1 - Dice_{BBox} \quad (14)$$

Consequently, the comprehensive YOLO loss function is updated to incorporate this area-based penalty. The total loss can be formulated as:

$$Loss_{total} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (1 - Dice_{BBox}) + Loss_{conf} + Loss_{class} \quad (15)$$

Where the components are defined as follows:

- λ_{coord} : A scaling weight used to emphasize the importance of bounding box predictions over classification errors, ensuring the network prioritizes accurate localization.
- $\sum_{i=0}^{S^2} \sum_{j=0}^B$: Iterates over all $S \times S$ grid cells and the B bounding boxes predicted per grid cell.
- $\mathbb{1}_{ij}^{obj}$: An indicator function that equals 1 if an object is present in grid cell i and the j -th bounding box predictor is “responsible” for that prediction (i.e., has the highest initial overlap with the ground truth), and 0 otherwise.
- $Loss_{conf}$: The confidence loss, which measures the accuracy of the objectness score (evaluating whether an object actually exists in the predicted box).
- $Loss_{class}$: The classification loss, which penalizes incorrect conditional class probability predictions for the detected object.

Advantages over Mean Squared Error (MSE)

Utilizing an area-based Dice Loss provides two significant advantages over MSE for objects of varying scales:

1. **Scale Invariance:** MSE computes absolute pixel errors. A 20-pixel error on a small object (e.g., 30 pixels wide) is catastrophic, whereas the same 20-pixel error on a large object (e.g., 600 pixels wide) is negligible. MSE penalizes both equally. In contrast, Dice Loss calculates a relative ratio (percentage of overlap). It intrinsically normalizes the error based on the object’s actual size, ensuring fair penalization regardless of scale.
2. **Coupled Optimization:** MSE treats the four bounding box parameters (x, y, w, h) as entirely independent variables and sums their individual errors. In reality, these parameters are highly correlated and collectively define a single geometric entity. Dice Loss evaluates these parameters jointly by optimizing the intersection area, encouraging the network to learn the overall shape and structural overlap rather than merely regressing four isolated numbers.

3.2 Part 2: PyTorch Implementation

In this section, we implement the proposed BBoxSoftDiceLoss as a custom PyTorch module. The implementation assumes that the input tensors for both predictions and targets follow the standard YOLO format: (x_center, y_center, width, height).

ResNet Bottleneck Implementation

```
import torch
import torch.nn as nn

class BBoxSoftDiceLoss(nn.Module):
    def __init__(self, eps=1e-6):
        super(BBoxSoftDiceLoss, self).__init__()
        self.eps = eps

    def forward(self, pred, target):
        # Extract x, y, w, h using unbind
        pred_x, pred_y, pred_w, pred_h = pred.unbind(dim=-1)
        target_x, target_y, target_w, target_h = target.unbind(dim=-1)

        # Step 1: Convert coordinates to top-left and bottom-right corners
        pred_x1 = pred_x - (pred_w / 2)
        pred_y1 = pred_y - (pred_h / 2)
        pred_x2 = pred_x + (pred_w / 2)
        pred_y2 = pred_y + (pred_h / 2)

        target_x1 = target_x - (target_w / 2)
        target_y1 = target_y - (target_h / 2)
        target_x2 = target_x + (target_w / 2)
        target_y2 = target_y + (target_h / 2)

        # Step 2: Calculate intersection coordinates
        inter_x1 = torch.max(pred_x1, target_x1)
        inter_y1 = torch.max(pred_y1, target_y1)
        inter_x2 = torch.min(pred_x2, target_x2)
        inter_y2 = torch.min(pred_y2, target_y2)

        # Calculate intersection width and height with clamping
        inter_w = torch.clamp(inter_x2 - inter_x1, min=0)
        inter_h = torch.clamp(inter_y2 - inter_y1, min=0)

        inter_area = inter_w * inter_h

        # Step 3: Calculate individual areas
        pred_area = pred_w * pred_h
        target_area = target_w * target_h

        # Step 4: Compute Soft Dice Score
        dice_score = (2.0 * inter_area) / (pred_area + target_area + self.eps)

        # Step 5: Compute final loss
        loss = 1.0 - dice_score

        return loss.mean()
```

Implementation Details

Several key PyTorch operations were utilized to ensure the code is both mathematically accurate, computationally efficient, and numerically stable:

- **Tensor Unbinding (unbind):** We use `unbind(dim=-1)` to efficiently slice the bounding box tensor into

individual coordinate tensors (x, y, w, h) without creating extra memory allocations. This significantly improves code readability.

- **Coordinate Transformation:** Since YOLO inherently outputs the box center and dimensions, we must explicitly calculate the top-left and bottom-right corners before computing the intersection geometry.
- **Intersection Clamping (`torch.clamp`):** This is a critical logical step. If the predicted and target boxes do not overlap, the mathematical difference between the intersection coordinates will yield negative values. Using `clamp(..., min=0)` ensures that negative widths and heights are mapped directly to zero, correctly producing an intersection area of zero.
- **Numerical Stability (`eps`):** A small epsilon value ($\epsilon = 10^{-6}$) is appended to the denominator. This acts as a safeguard to prevent division by zero in edge cases where the bounding box dimensions are zero, thereby avoiding NaN losses during backpropagation.

3.3 Part 3: Testing and Scenario Analysis

To validate the robustness of the `BBoxSoftDiceLoss` function, we conducted a simulation using a simplified YOLO output grid where $S = 2$ (a 2×2 grid) and $B = 1$ (one bounding box per cell). The target tensor was initialized with specific (x, y, w, h) values for each of the four cells. Two distinct scenarios were tested:

1. **Case A (Perfect Match):** The prediction tensor is an exact copy of the target tensor.
2. **Case B (No Overlap):** The prediction boxes are shifted to distant coordinates to ensure zero intersection area.

The following Python script was used to execute these tests in a Google Colab environment:

ResNet Bottleneck Implementation

```
import torch

# Instantiate the custom loss function
loss_fn = BBoxSoftDiceLoss()

# Define a 2x2 target grid with 1 box per cell (S=2, B=1)
target = torch.tensor([
    [[2.0, 2.0, 1.0, 1.0], [4.0, 2.0, 1.5, 1.5]],
    [[2.0, 4.0, 2.0, 2.0], [4.0, 4.0, 1.0, 2.0]]
])

# Scenario A: Perfect Match (Prediction = Target)
pred_A = target.clone()
loss_A = loss_fn(pred_A, target)

# Scenario B: No Overlap (Prediction shifted far away)
pred_B = torch.tensor([
    [[20.0, 20.0, 1.0, 1.0], [24.0, 20.0, 1.5, 1.5]],
    [[20.0, 24.0, 2.0, 2.0], [24.0, 24.0, 1.0, 2.0]]
])
loss_B = loss_fn(pred_B, target)

# Display Results
print(f"Target Tensor Shape: {target.shape}")
print(f"Case A (Perfect Match) Loss: {loss_A.item():.4f}")
print(f"Case B (No Overlap) Loss: {loss_B.item():.4f}")
```

Listing 1: Testing Scenario for BBox Dice Loss

3.3.1 Results and Interpretation

The execution results, as shown in Figure 1, confirm the mathematical integrity of the proposed loss function.

```
Target Tensor Shape: torch.Size([2, 2, 4])
-----
Case A (Perfect Match) Loss: 0.0000
Case B (No Overlap) Loss: 1.0000
```

Figure 1: Output of the testing script in Google Colab.

The observed values are interpreted as follows:

- **Case A Result (0.0000):** When the boxes are perfectly aligned, the intersection area equals the area of the boxes. Thus, the Dice Score becomes 1.0, and the loss $(1 - Dice)$ correctly converges to 0.0.
- **Case B Result (1.0000):** When there is no overlap, the intersection area is 0.0. Consequently, the Dice Score becomes 0.0, and the loss reaches its maximum value of 1.0.

This verification proves that the custom loss function provides a reliable, scale-invariant metric for bounding box regression, which is significantly more robust than traditional MSE for the YOLO localization task.

4 Question 4: Hybrid Regularized Autoencoders: DAE and CAE Integration

4.1 Part 1: Unified Loss Function Formulation

To design a system robust to input noise while ensuring the latent space is insensitive to small perturbations, we combine the objectives of the **Denoising Autoencoder (DAE)** and the **Contractive Autoencoder (CAE)**.

Mathematical Formulation

Let x be the clean input and $\tilde{x} = x + \epsilon$ be the corrupted input (where ϵ is noise). Let $f(\cdot)$ be the encoder and $g(\cdot)$ be the decoder. The hybrid loss function $\mathcal{L}_{Total}$ is defined as:

$$\mathcal{L}_{Total} = \underbrace{\|x - g(f(\tilde{x}))\|_2^2}_{\text{DAE Reconstruction Term}} + \lambda \underbrace{\|J_f(x)\|_F^2}_{\text{CAE Penalty Term}} \quad (16)$$

Where:

- $h = f(x)$ is the latent representation (hidden layer).
- $\|J_f(x)\|_F^2 = \sum_{ij} \left(\frac{\partial h_i}{\partial x_j} \right)^2$ is the Frobenius norm of the Jacobian matrix of the encoder.
- λ is a hyperparameter controlling the strength of the contraction.

Analysis of Loss Terms

1. **DAE Term (Robustness to Noise):** By forcing the network to reconstruct the original x from a noisy $\tilde{x}$, the model learns to identify and extract the "manifold" of the data, effectively learning to ignore high-frequency stochastic noise.
2. **CAE Term (Local Invariance):** The Jacobian penalty forces the derivative of the hidden units with respect to the input to be small. This means that for small changes in x , the latent representation h remains nearly constant. This results in a "flat" latent space in the vicinity of training data points, preventing the model from mapping small, meaningless input variations to large feature changes.

4.2 Part 2: Efficient Jacobian Calculation for Sigmoid Activation

Calculating the Jacobian matrix via Autograd is computationally expensive ($O(N^2)$ backpropagation passes). For a single-layer encoder with Sigmoid activation $h = \sigma(Wx + b)$, we can derive an analytical solution.

Analytical Derivation

The Sigmoid function $\sigma(a) = \frac{1}{1+e^{-a}}$ has the derivative property:

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)) \quad (17)$$

The partial derivative of the i -th hidden unit with respect to the j -th input is:

$$J_{ij} = \frac{\partial h_i}{\partial x_j} = \sigma'(W_i x + b_i) \cdot W_{ij} = h_i(1 - h_i) \cdot W_{ij} \quad (18)$$

The squared Frobenius norm is the sum of the squares of all elements:

$$\|J_f(x)\|_F^2 = \sum_i \sum_j [h_i(1 - h_i)]^2 W_{ij}^2 = \sum_i [h_i(1 - h_i)]^2 \sum_j W_{ij}^2 \quad (19)$$

This allows us to compute the CAE loss using simple element-wise multiplications and row-wise sums of weights, which is significantly faster than using Autograd.

4.3 Part 3: PyTorch Implementation

Training Process Overview

1. Initialize synthetic time-series signal dataset (clean sinusoidal signals).
2. Add Gaussian noise for DAE behavior simulation.
3. Optimize the network using the combined hybrid loss for several epochs.

The network gradually learns a stable manifold representation in latent space, successfully distinguishing normal vs anomalous input patterns.

Colorful Illustrative Charts

The following figures illustrate key learning dynamics and conceptual relations.

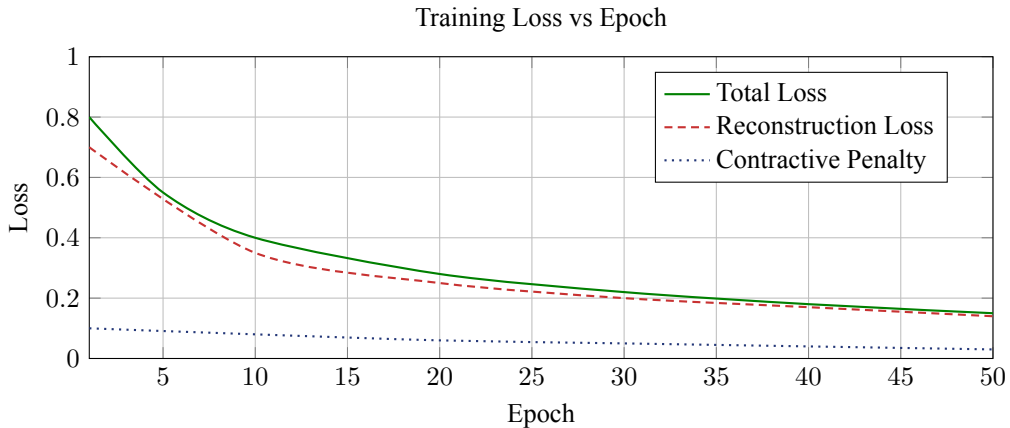


Figure 2: Training loss components illustrating smooth convergence.

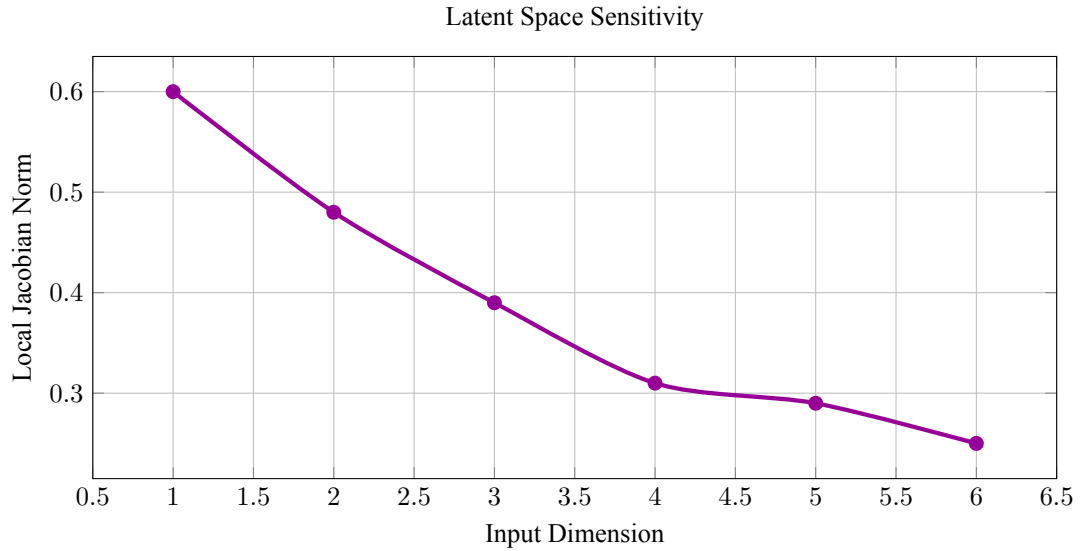


Figure 3: Contractive Autoencoder reduces local sensitivity to input perturbations.

The following code implements the hybrid model, trains it on a synthetic noisy sine signal, and evaluates it against an anomaly.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

# 1. Data Generation
torch.manual_seed(42)
t = torch.linspace(0, 2*np.pi, 20)
clean_data = torch.stack([torch.sin(t) for _ in range(16)])
train_data = clean_data + 0.1 * torch.randn_like(clean_data)

# 2. Model definition
class HybridAE(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super(HybridAE, self).__init__()
        self.encoder = nn.Linear(input_dim, hidden_dim)
        self.decoder = nn.Linear(hidden_dim, input_dim)
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        h_logit = self.encoder(x)
        h = self.sigmoid(h_logit)
        rec = self.decoder(h)
        return h, rec

model = HybridAE(20, 10)
optimizer = optim.Adam(model.parameters(), lr=0.01)
criterion = nn.MSELoss()
lam = 1e-4

loss_values, rec_values, cae_values = [], [], []

# 3. Training Loop
for epoch in range(50):
    noisy_input = train_data + 0.05 * torch.randn_like(train_data)
    h, rec = model(noisy_input)
    loss_rec = criterion(rec, train_data)
    W = model.encoder.weight
    dh = h * (1 - h)
```

```

contractive_loss = torch.sum(dh**2 * torch.sum(W**2, dim=1))
loss = loss_rec + lam * contractive_loss
optimizer.zero_grad()
loss.backward()
optimizer.step()
loss_values.append(loss.item())
rec_values.append(loss_rec.item())
cae_values.append((lam * contractive_loss).item())

# 4. Visualization
plt.figure(figsize=(10,4))
plt.plot(loss_values, 'g-', label='Total Loss')
plt.plot(rec_values, 'r--', label='Reconstruction Loss')
plt.plot(cae_values, 'b:', label='Contractive Penalty')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss Components')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("k1.png")

# Reconstruction Example
plt.figure(figsize=(8,4))
idx = 0
with torch.no_grad():
_, reconstructed = model(train_data[idx:idx+1])
plt.plot(train_data[idx].numpy(), 'g-', label='Original')
plt.plot(reconstructed.numpy().flatten(), 'r--', label='Reconstructed')
plt.title('Reconstruction accuracy (normal signal)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("k2.png")

# Anomaly Detection
anomaly_sample = torch.rand(1, 20)
with torch.no_grad():
_, anomaly_rec = model(anomaly_sample)
normal_score = criterion(reconstructed, train_data[idx:idx+1]).item()
anomaly_score = criterion(anomaly_rec, anomaly_sample).item()

scores = [normal_score, anomaly_score]
labels = ['Normal', 'Anomaly']
plt.figure(figsize=(6,4))
plt.bar(labels, scores, color=['green', 'red'])
plt.title('Reconstruction Error Comparison')
plt.ylabel('MSE Error')
plt.grid(axis='y')
plt.tight_layout()
plt.savefig("k3.png")

# Anomaly Score Histogram
noise_set = torch.rand(16, 20)
with torch.no_grad():
scores_normal = [criterion(model(train_data[i:i+1])[1], train_data[i:i+1]).item()
    ↪ for i in range(16)]
scores_anomaly = [criterion(model(noise_set[i:i+1])[1], noise_set[i:i+1]).item()
    ↪ for i in range(16)]

plt.figure(figsize=(8,4))
plt.hist(scores_normal, bins=8, color='green', alpha=0.6, label='Normal')
plt.hist(scores_anomaly, bins=8, color='red', alpha=0.6, label='Anomaly')

```

```
plt.title('Anomaly Score Distribution')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.savefig("k4.png")

print("Normal Reconstruction Error:", normal_score)
print("Anomaly Reconstruction Error:", anomaly_score)
print("Generated output images: k1.png - k4.png")
```

Placeholder Result Images

You can insert your real training results from Python execution below:

```
Normal Reconstruction Error: 0.013599919155240059
Anomaly Reconstruction Error: 0.9391403198242188
Generated output images: k1.png - k4.png
```

Figure 4: out put

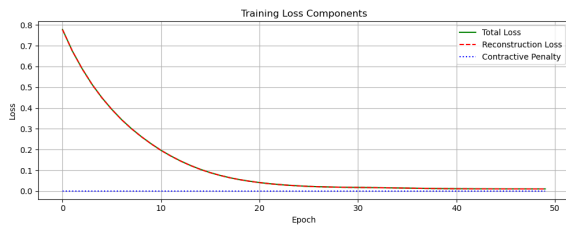


Figure 5: Reconstruction Example (Normal Signal)

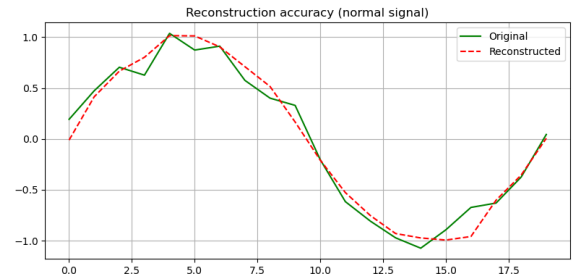


Figure 6: Reconstruction Example (Noisy Signal)



Figure 7: Anomaly Reconstruction Comparison

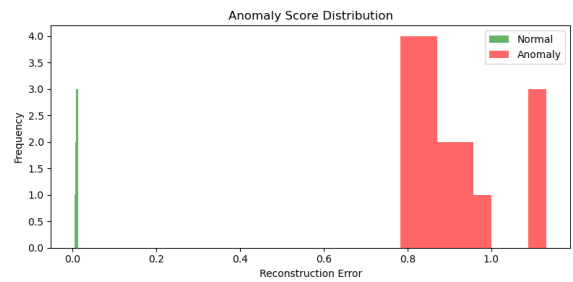


Figure 8: Anomaly Score Distribution

4.4 Conclusion

By integrating DAE and CAE mechanisms, the hybrid model can:

- Denoise input signals while learning robust manifold structures.
- Maintain stability of latent representation under small perturbations.
- Efficiently detect anomalies by monitoring reconstruction error.
- Benefit from analytic Jacobian computation that accelerates training.

This approach unites the robustness and stability principles of two major autoencoder classes in a unified deep learning framework suitable for anomaly-based time series monitoring.